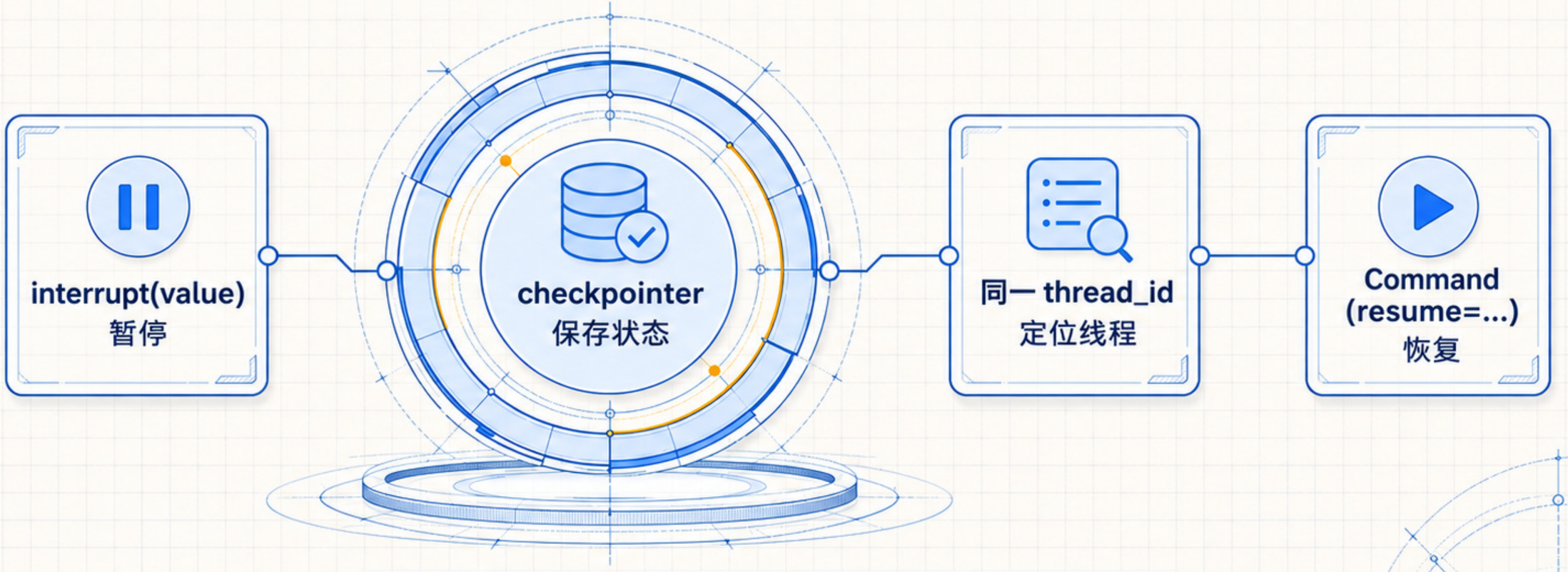
Deep Agents 实战

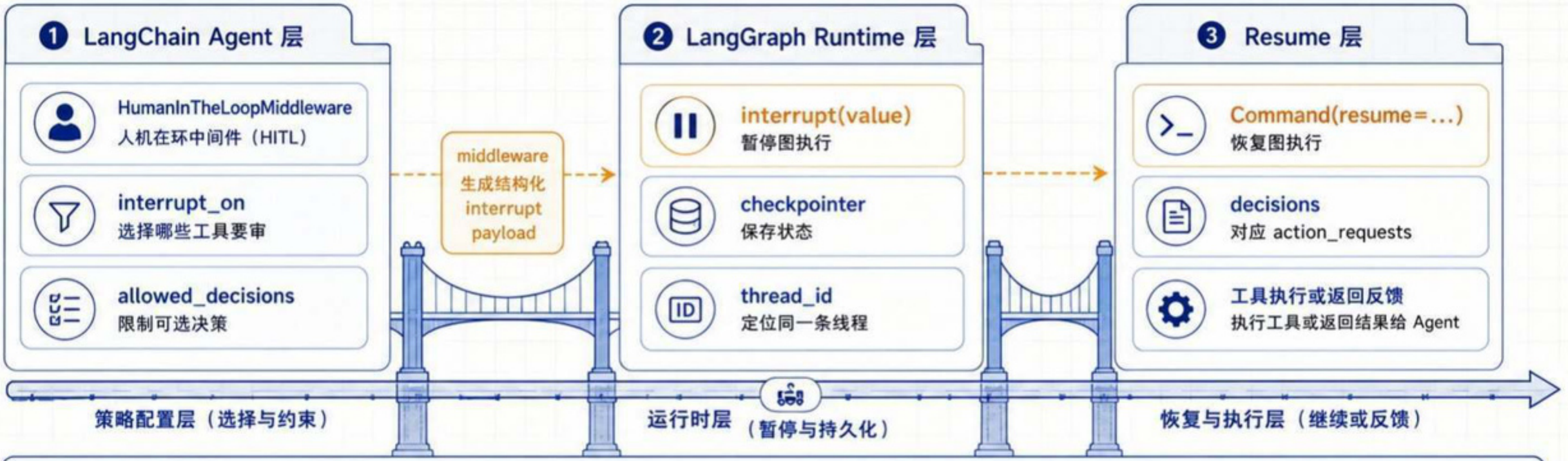
HITL 进阶 — 中断、重放与审批边界



不是恢复 Python 调用栈，而是重新进入节点

LangGraph 原语 × LangChain 中间件

HITL middleware 是工具调用层的 interrupt 封装




```
interrupt.value = {
  "action_requests": [...],
  "review_configs": [...],
}
```

```
agent.invoke(Command(resume={"decisions": [...]}),
             config=config, version="v2")
```

底层只负责暂停与恢复

中间件负责工具审批语义

直接调用 interrupt()

为业务节点收集人工决策

```python
from langgraph.types import interrupt

def review_node(state):
    decision = interrupt({
        "draft": state["draft"],
        "question": "是否批准？",
    })

    return {"approved": decision["approved"]}
```

审查载荷 → 人工决策 → 返回节点状态

- 在已编译图的节点中运行；
- 图需配置 checkpointer；
- 恢复时复用同一 thread_id。

示例是业务节点，不是工具审批的 decisions 协议。

恢复值要匹配暂停点

两种协议，不要混用

底层 interrupt()：返回自定义值

```
graph.invoke(
    Command(resume={"approved": True}),
    config=config,
)
```

approved 是示例业务字段；拒绝时改为 False，而非连续提交两次。

interrupt_on：工具审批协议

```
agent.invoke(
    Command(resume={
        "decisions": [{"type": "approve"}]
    }),
    config=config,
)
```

decisions 与待审工具顺序、数量对应。

两种恢复都需要同一线程与可用 checkpoint；示例为两种独立场景。

排查暂停在哪里

用 get_state 看 values、next、interrupts

```python
snapshot = graph.get_state(config)

print(snapshot.values)
print(snapshot.next)
print(snapshot.interrupts)


# Platform / SDK
thread_state = client.threads.get_state(
    thread_id="thread-1")
```





恢复时会从节点开头重放

不是从 interrupt() 下一行继续




```python
def node(state):
    before_interrupt()  # resume 后会再跑
    answer = interrupt("是否继续？")
    after_interrupt(answer)
    return state
```

节点开头的代码会再次执行
不会跳过

在此处暂停，等待外部确认

确认后继续执行后续逻辑

规则 1：不要裸 try/except

不要吞掉 interrupt 的特殊异常

错误：裸 except 会吞掉中断

```
try:
    result = interrupt("请审批")
except Exception as e:
    print(e)
```

Exception 会误捕获特殊异常

interrupt 需要向外冒泡

正确：只捕获业务异常

```
try:
    result = interrupt("请审批")
    fetch_data()
except NetworkError as e:
    print(e)
```

只捕获 NetworkError

副作用：审批之后也要防重

恢复会重放节点，不能承诺 exactly-once

避免审批前写入

```python
def node(state):
    db.create_log("开始")
    approved = interrupt("请审批")
```

恢复时可能重复写入。

审批后 + 稳定幂等键

```python
def node(state):
    approved = interrupt("请审批")
    if approved:
        db.upsert(state["operation_id"], "已批准")
    return {"approved": approved}
```

使用稳定的操作 ID 进行幂等写入。

`db.upsert` 是伪代码；用可信、稳定的操作 ID，并由存储层保证幂等。放在审批后避免未授权写入，但重试或后续中断仍可能重放。

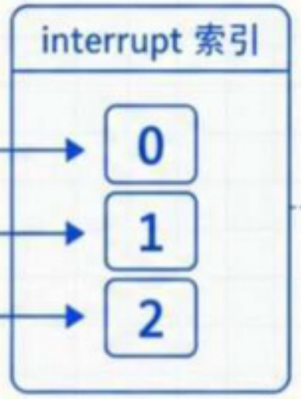
规则 3：不要改变调用顺序

同节点多个 interrupt 按索引匹配

Good

顺序固定，
索引匹配

```
1  name = interrupt("姓名？")
2  age  = interrupt("年龄？")
3  city = interrupt("城市？")
```

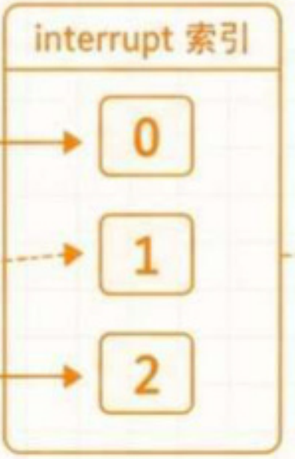


调用顺序稳定，
索引与问题一一对应，
不会错配。

Bad

条件跳过，
导致错配

```
1  name = interrupt("姓名？")
2  if state.get("need_age"):
3      age = interrupt("年龄？")
4  city = interrupt("城市？")
```



当 need_age 为 False 时，
会跳过第 1 个 interrupt。

后续索引前移，
导致与之前保存的索引错配，
恢复对话将出现错误。

规则 4：并行中断用 ID 恢复

界面可排序，恢复值不能错配

```
resume = {
    intr.id: ui_answers[intr.id]
    for intr in stream.interrupts
}

graph.invoke(Command(resume=resume), config=config)
```

用 id 做 key

按 id 取界面答案

排序变化也不影响恢复

审批卡片 A

Interrupt.id = "a17"

请审核申请 A17

同意

审批卡片 B

Interrupt.id = "b42"

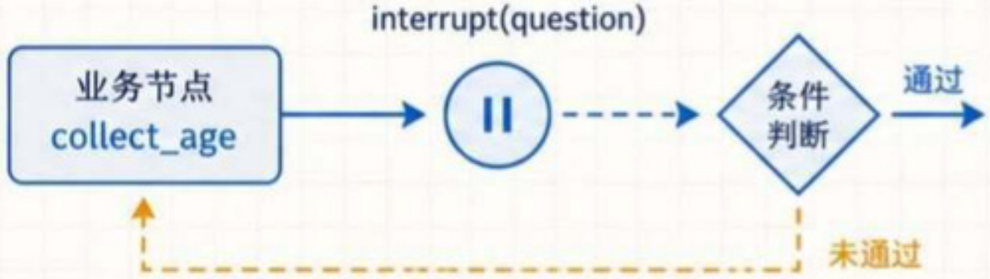
请审核申请 B42

同意

输入验证与静态中断

业务输入用状态回路，调试断点用静态中断

业务输入验证：单次 interrupt + 条件边

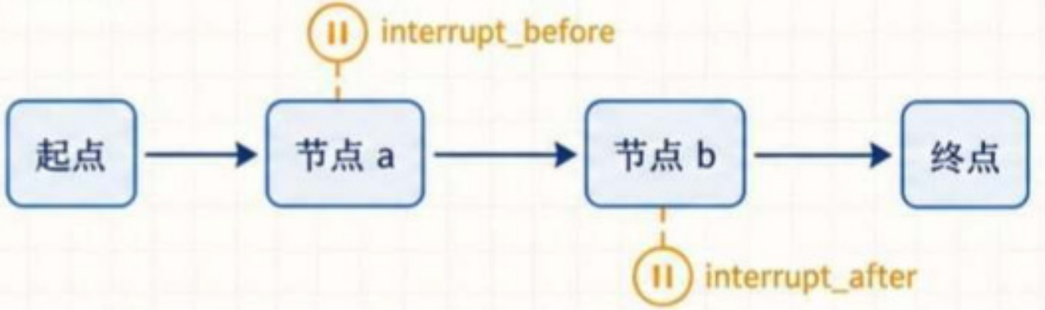



```
answer = interrupt(question)
return {"pending_question": next_question}
```

中断并等待
用户输入

未通过则回到
业务节点重试

调试断点：compile(interrupt_before/after)




```
graph = builder.compile(
    interrupt_before=["node_a"],
    interrupt_after=["node_b"],
    checkpointer=checkpointer,
)
```

在 node_a 之前
暂停

在 node_b 之后
暂停

持久化与恢复
由 checkpointer 管理

Middleware 中断：选择清晰的边界

不对应单个工具的审查，也能注入 Harness

模型节点
生成草稿

→

after_model 节点
检查无 tool_calls 后 interrupt

→

人工审批

→

返回图继续

Node-style Hook

- before_model
- after_model
- before_agent
- after_agent

独立图节点，适合人工暂停。

Wrap-style Hook

- wrap_model_call
- wrap_tool_call

在 model/tools 节点内部，适合重试、缓存、转换。

⚠ 在 handler() 后中断，恢复时可能重新执行模型或工具；checkpointer 不提供副作用幂等保证。

最终草稿审查：先暂停，再交付

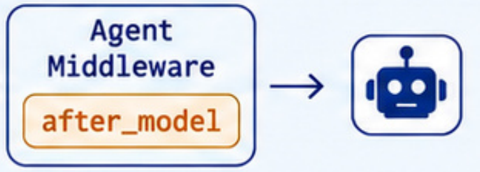
after_model 中过滤工具调用，避免把每轮推理都当最终答复

```python
last = state["messages"][-1]
if not isinstance(last, AIMessage) \
    or last.tool_calls:
    return None

choice = interrupt({"draft": last.content})
if choice.get("approved"):
    return None

return {"messages": [AIMessage(content="草稿未发布")]}
```

1. 放入 AgentMiddleware.after_model；



2. 通过 middleware=[...] 注入；



3. 配置 checkpointer 并在同一 thread_id 恢复。



伪代码省略导入与类定义；完整结构见源文章。

这是最终结果交付门；若客户端提前展示原始 token 流，仍须在展示层隔离未审批草稿。
审批不等于自动发布到外部渠道。

本讲回顾：安全恢复靠明确边界

1 暂停协议



`interrupt()`

`interrupt_on`

- interrupt() 自定义值；
- interrupt_on 用 decisions。

2 定位与存储

`thread_id` + 

- 同一 thread_id + checkpointer；
- 跨进程需要持久存储。

3 重放



- 节点从头执行，副作用要幂等。

4 匹配





- 同节点顺序稳定；
- 并行中断用 ID。

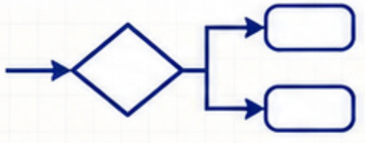
5 生命周期





- Node-style 人工暂停；
- Wrap-style 控制调用。

6 调试与输入



`None`

- 条件边收集输入；
- 静态断点用 None 继续。



下一章：Sandboxes —— 在受控环境中执行代码

